

CORE PLATFORM FRAMEWORK

OFFICIAL PRODUCT DOCUMENTATION

Build · Run · Recover · Audit

master 64fd08d96392 | source 4c4248a12e69 | 186 requirements



CORE PLATFORM FRAMEWORK

아키텍처설계서

제품 구조 · Boundary · Topology · Transaction · Security · Operations

항목	기준
대상	아키텍트, Tech Lead, 제품/플랫폼 설계자, 검토자
정보 역할	Architecture Decision 과 구현 경계 정보
기준 Repository	freeangelsun/202412_01_CPF · master
기준 Commit	64fd08d963927860e8d023403dfa276931801ee5
요구사항 정보	CPF_FINAL_TARGET_REQUIREMENTS.md · 186 개
현행화 Source	4c4248a12e699c07f9f5fb11fbb33b97ca04077d

이 문서는 현재 구현의 상태 보고가 아니라, CPF 제품 사용자가 업무를 끝내기 위해 따라야 할 완성 기준 사용 계약을 중심으로 작성한다. 실제 CPF Symbol/Path 는 CURRENT SOURCE, 구현 독립 예시는 REFERENCE 로 구분한다.

문서 읽는 법과 사실 구분

표기	의미	사용 규칙
CURRENT SOURCE	최신 master 에서 직접 확인한 실제 Class/API/Route/Property/SQL/Test/경로	복사·검색 가능한 실제 이름으로 사용한다.
PRODUCT CONTRACT	CPF 가 완료된 제품으로서 제공해야 하는 사용·운영 계약	현재 구현량 때문에 축소하지 않는다.
REFERENCE	사용자가 그대로 응용할 수 있도록 제공하는 완성 기준 코드·설정·절차 예시	실제 CPF 내부 Symbol 인 것처럼 표현하지 않는다.

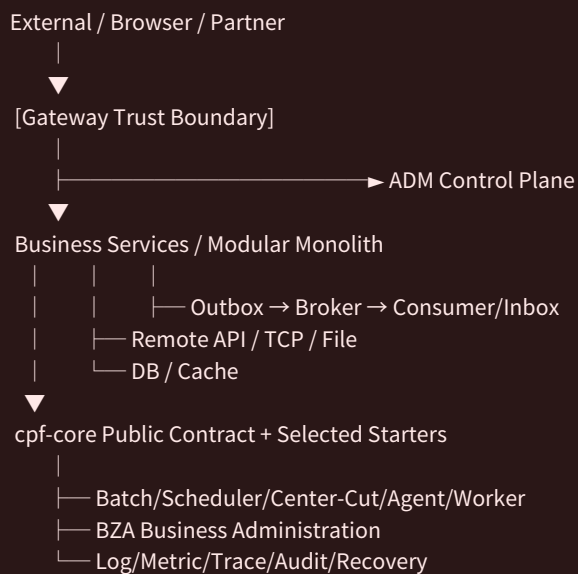
본문은 기능을 찾는 데서 끝나지 않는다. 적용 가능한 범위에서 선택 기준 → 준비 → 구현/설정 → 실행 → 정상 결과 → 실패/부분 실패 → 복구·대사 → 보안·감사 → Test → 운영 인계 순서로 설명한다.

1. Architecture Drivers

Driver	Architecture consequence
Consistency	Header/Error/Security/Transaction 의미를 Domain 마다 재구현하지 않음
Reliability	timeout/duplicate/partial/UNKNOWN/process kill 을 1 급 상태로 다룸
Operability	관리자가 transactionId 하나로 lineage 를 찾고 중복·오처리 없이 조치
Extensibility	Public API/SPI + Starter/Provider + Generator 로 확장
Portability	Local/Remote, JAR/WAR/Static/Worker, 3 DB Vendor
Security	trust boundary, least privilege, masking, approval, tamper-evident audit
Recoverability	Retry/Restart/Reprocess/Reconcile/Compensation/Rollback/Restore/DR

2. Context Architecture

● ● ● FLOW · 2. Context Architecture



3. Module Ownership

Owner	Owns	Must not own
cpf-core	CPF 전역 Kernel / Contract / Semantics	특정 Owner 전용 API · SPI, Optional Capability 계약, Spring/Logging/JDBC/JPA/MyBatis Runtime
cpf-common	고객 공통 업무 정책	기술 Engine/특정 Domain
cpf-starters	Provider/capability runtime	업무 Domain
cpf-gateway	외부 ingress/data plane/control integration	내부 service bus
cpf-admin	platform operations control plane	타 Owner DB 직접 수정
cpf-biz-admin	조직/권한/업무 결재	platform runtime control
cpf-batch	Spring Batch/Scheduler/Center-Cut/Agent/Worker	중복 Job engine
cpf-tools	Generator/BOM/DB vendor/build/deploy tooling	runtime business state

3.1 Dependency Rules

- Core 는 Business/Admin/Batch/Gateway 구체 구현에 의존하지 않는다.
- 업무 Domain 은 다른 Domain DB/Table/Repository 를 직접 접근하지 않는다.
- ADM/BZA 는 Owner Query/Command 를 사용한다.
- Provider 선택을 Core type signature 에 강제하지 않는다.
- Starter aggregate 는 leaf bean 을 중복 소유하지 않는다.
- 고객 extension 은 Public API/SPI 만 소비하며 Internal package compile 을 금지한다.

4. Starter Architecture

● ● ● CODE · 4. Starter Architecture

```
cpf-core (Boot-free contract)
↓
Foundation Leaf Starters
↓
Capability Leaf Starters
(data / messaging / integration / file / notification / security / operations)
↓
Generator Profile Resolver
↓ resolvedStarters + versions + lock
Aggregate/Profile dependency-only entry
↓
Customer/Generated Domain
```

Public Profile	Architecture intent
minimal-domain	최소 footprint
web-api	HTTP API
secure-api	resource security
browser-bff	server session/BFF
event-service	broker/event
batch-service	batch workload

5. Runtime Topology

Modular Monolith

● ● ● FLOW · Modular Monolith

Channel → Local Facade → Application/Domain → DB

업무 계약은 topology-neutral 하다. Network 에만 존재하는 timeout/circuit/retry 는 Remote Adapter 에서 추가하지만 오류 의미, authorization, idempotency, audit 와 transaction lineage 를 바꾸지 않는다.

Microservices

● ● ● FLOW · Microservices

Gateway → Service A → Remote Facade → Service B → DB

업무 계약은 topology-neutral 하다. Network 에만 존재하는 timeout/circuit/retry 는 Remote Adapter 에서 추가하지만 오류 의미, authorization, idempotency, audit 와 transaction lineage 를 바꾸지 않는다.

Hybrid

● ● ● CODE · Hybrid

일부 Domain Same-JVM + 일부 Remote

업무 계약은 topology-neutral 하다. Network 에만 존재하는 timeout/circuit/retry 는 Remote Adapter 에서 추가하지만 오류 의미, authorization, idempotency, audit 와 transaction lineage 를 바꾸지 않는다.

Batch

● ● ● FLOW · Batch

Scheduler → Job/Step → Business Facade → Resource

업무 계약은 topology-neutral 하다. Network 에만 존재하는 timeout/circuit/retry 는 Remote Adapter 에서 추가하지만 오류 의미, authorization, idempotency, audit 와 transaction lineage 를 바꾸지 않는다.

Async

● ● ● FLOW · Async

Domain → Outbox → Broker → Inbox/Dedup → Domain

업무 계약은 topology-neutral 하다. Network 에만 존재하는 timeout/circuit/retry 는 Remote Adapter 에서 추가하지만 오류 의미, authorization, idempotency, audit 와 transaction lineage 를 바꾸지 않는다.

6. Transaction & Consistency Architecture

LOCAL

항목	내용
목적	single resource ACID boundary
대상 역할	architect/developer/operator
Owner	core contract + runtime owner
실제/대표 Consumer	business domain/integration/batch
선행 조건	provider and durability prerequisites
입력·기본값·범위	deadline/idempotency/version
정상 흐름	single resource ACID boundary
정상 판정	terminal state and resource effects agree
오류·경계·부분 실패	timeout/kill/duplicate/partial
복구·대사·Rollback	recovery/reconcile/compensation
권한·보안·감사	least privilege/masking/approval/audit
Log·Metric·Trace	transactionId + segment/attempt + metrics
Test	fault/restart/multi-instance
운영 인계	ADM Timeline/Recovery

XA/JTA

항목	내용
목적	Transaction Manager + 2PC + XA resources + recovery log
대상 역할	architect/developer/operator
Owner	core contract + runtime owner
실제/대표 Consumer	business domain/integration/batch
선행 조건	provider and durability prerequisites
입력·기본값·범위	deadline/idempotency/version
정상 흐름	Transaction Manager + 2PC + XA resources + recovery log
정상 판정	terminal state and resource effects agree
오류·경계·부분 실패	timeout/kill/duplicate/partial
복구·대사·Rollback	recovery/reconcile/compensation
권한·보안·감사	least privilege/masking/approval/audit
Log·Metric·Trace	transactionId + segment/attempt + metrics
Test	fault/restart/multi-instance
운영 인계	ADM Timeline/Recovery

OUTBOX

항목	내용
목적	business DB transaction + durable outbox + publisher
대상 역할	architect/developer/operator
Owner	core contract + runtime owner
실제/대표 Consumer	business domain/integration/batch
선행 조건	provider and durability prerequisites
입력 · 기본값 · 범위	deadline/idempotency/version
정상 흐름	business DB transaction + durable outbox + publisher
정상 판정	terminal state and resource effects agree
오류 · 경계 · 부분 실패	timeout/kill/duplicate/partial
복구 · 대사 · Rollback	recovery/reconcile/compensation
권한 · 보안 · 감사	least privilege/masking/approval/audit
Log · Metric · Trace	transactionId + segment/attempt + metrics
Test	fault/restart/multi-instance
운영 인계	ADM Timeline/Recovery

INBOX_DEDUP

항목	내용
목적	durable consumer ledger before/with business effect
대상 역할	architect/developer/operator
Owner	core contract + runtime owner
실제/대표 Consumer	business domain/integration/batch
선행 조건	provider and durability prerequisites
입력 · 기본값 · 범위	deadline/idempotency/version
정상 흐름	durable consumer ledger before/with business effect
정상 판정	terminal state and resource effects agree
오류 · 경계 · 부분 실패	timeout/kill/duplicate/partial
복구 · 대사 · Rollback	recovery/reconcile/compensation
권한 · 보안 · 감사	least privilege/masking/approval/audit
Log · Metric · Trace	transactionId + segment/attempt + metrics
Test	fault/restart/multi-instance
운영 인계	ADM Timeline/Recovery

SAGA

항목	내용
목적	durable orchestrated/choreographed steps + compensation
대상 역할	architect/developer/operator
Owner	core contract + runtime owner
실제/대표 Consumer	business domain/integration/batch
선행 조건	provider and durability prerequisites
입력 · 기본값 · 범위	deadline/idempotency/version
정상 흐름	durable orchestrated/choreographed steps + compensation
정상 판정	terminal state and resource effects agree
오류 · 경계 · 부분 실패	timeout/kill/duplicate/partial
복구 · 대사 · Rollback	recovery/reconcile/compensation
권한 · 보안 · 감사	least privilege/masking/approval/audit
Log · Metric · Trace	transactionId + segment/attempt + metrics
Test	fault/restart/multi-instance
운영 인계	ADM Timeline/Recovery

TCC

항목	내용
목적	reservation lifecycle Try/Confirm/Cancel
대상 역할	architect/developer/operator
Owner	core contract + runtime owner
실제/대표 Consumer	business domain/integration/batch
선행 조건	provider and durability prerequisites
입력 · 기본값 · 범위	deadline/idempotency/version
정상 흐름	reservation lifecycle Try/Confirm/Cancel
정상 판정	terminal state and resource effects agree
오류 · 경계 · 부분 실패	timeout/kill/duplicate/partial
복구 · 대사 · Rollback	recovery/reconcile/compensation
권한 · 보안 · 감사	least privilege/masking/approval/audit
Log · Metric · Trace	transactionId + segment/attempt + metrics
Test	fault/restart/multi-instance
운영 인계	ADM Timeline/Recovery

UNKNOWN/RECONCILE

항목	내용
목적	uncertain external outcome + result query + operator workflow
대상 역할	architect/developer/operator
Owner	core contract + runtime owner
실제/대표 Consumer	business domain/integration/batch
선행 조건	provider and durability prerequisites
입력 · 기본값 · 범위	deadline/idempotency/version
정상 흐름	uncertain external outcome + result query + operator workflow
정상 판정	terminal state and resource effects agree
오류 · 경계 · 부분 실패	timeout/kill/duplicate/partial
복구 · 대사 · Rollback	recovery/reconcile/compensation
권한 · 보안 · 감사	least privilege/masking/approval/audit
Log · Metric · Trace	transactionId + segment/attempt + metrics
Test	fault/restart/multi-instance
운영 인계	ADM Timeline/Recovery

7. Transaction Lineage Data Model

● ● ● TREE · 7. Transaction Lineage Data Model7. Transaction Lineage Data Mod

```
Transaction(root transactionId)
├─ Segment(segmentId,parentSegmentId,role,direction,instance,attempt)
│   ├── DB effect(resource,commit/rollback)
│   ├── RemoteCall(destination,requestId,attempt,outcome)
│   ├── Message(messageId,producer/consumer,partition,delivery)
│   ├── Batch(job/execution/step/partition/item/worker)
│   ├── File(transferId/checksum/state)
│   └─ Recovery(reconcile/compensate/reprocess/manual action)
```

Identity	Invariant
transactionId	root chain 전체에서 유지
segmentId	각 local/remote/message/batch continuation 구간
parentSegmentId	호출 계층
attempt	retry/reprocess 시 증가
trace/span	observability correlation
request/idempotency	업무 동일성/재시도
instance/worker/agent	실행 주체
job/execution/item	Batch identity

8. Failure Architecture

Failure	Architecture response
Validation/Auth	side effect 전 reject
Timeout before send	bounded retry 가능 여부 판단
Response loss	UNKNOWN + result query
Partial commit	resource 별 confirmed/unknown 을 ledger 로 보존
Duplicate	idempotency conflict/replay
Process kill	durable state + startup recovery
Stale worker	lease/fencing token reject
DB/Broker outage	backpressure/circuit/spool/retry cap
Compensation failure	UNKNOWN/MANUAL_REVIEW
Observability failure	policy 별 fail-open/closed + spool/terminal-loss

9. Data Architecture

Schema	Owner/목적
cpfDB	core platform canonical state
cmnDB	common
admDB	ADM
bzaDB	BZA
batDB	Batch
refDB	Reference
Generated Domain Schema	Generator manifest owner

MariaDB/PostgreSQL/Oracle 의 Install/Migration/Rollback/Runtime query 는 canonical schema/metadata 에서 같은 결과를 생성해야 하며 Vendor 단순 문자열 치환으로 간주하지 않는다.

10. Security Architecture

Trust Boundary

Gateway/channel 인증에서 내부 transaction/security header spoof 를 차단

Identity

user/operator/service MFA/OIDC/OAuth2/JWT/API key/mTLS

Authorization

RBAC/ABAC/Data Scope/SoD/server enforcement

Secret/Crypto

Vault/env/file + KMS/HSM/PKCS#11, keyId/version/rotation/revocation

Approval

ALL/ANY/N_OF_M, requester/approver 분리, expiry, command hash

Break-glass

별도 권한/TTL/emergency reason/post review

Audit

append-only/tamper-evident hash chain + optional digital signature

Privacy

PII classification/minimization/masking/raw approval/retention

11. ADM/BZA Architecture

ADM 은 platform owner 의 Query/Command 를 호출하는 Control Plane 이고, BZA 는 고객 업무 관리/조직/권한/결재 제품이다. 두 Approval Engine 의 정책/DB/책임을 섞지 않는다. Browser frontend 는 Vue3/TS/Vite, route registry, Pinia/TanStack Query, OpenAPI generated client exact-SHA drift gate 를 사용한다.

12. Gateway Architecture

Data Plane 과 Control Plane 을 분리하고, route snapshot 은 version/checksum 기반 atomic publish 한다. SSRF/trusted header/body replay/streaming/client disconnect/attempt ledger 를 ingress failure model 에 포함한다.

13. Batch Architecture

Spring Batch 가 primary execution engine 이며 CPF 는 approval/idempotency/fencing/unknown ledger 와 JobInstance/JobExecution/StepExecution 을 연결한다. Remote topology 는 durable transport 를 사용하고 in-memory fallback 을 제품 Profile 에서 허용하지 않는다.

14. Deployment & Supply Chain Architecture

● ● ● FLOW · 14. Deployment & Supply Chain Architecture14. Deployment & Suppl

Source → Build → Published POM/BOM/source/javadoc
→ Final JAR/WAR/static/worker artifact
→ Manifest + SHA-256 + signature/keyId
→ SBOM/license/vulnerability
→ Deploy request hash/version/sequence
→ Readiness + service identity + build SHA
→ Promotion / selective rollback / reconciliation

15. Architecture Decision Checklist

- Owner Module/Package/Public API/SPI/Internal
- Actual consumer and topology
- DB/schema/migration/rollback
- Transaction/failure/unknown/recovery
- Security/authorization/masking/approval/audit
- Multi-instance/lease/fence/rebalance
- Observability/SLO/incident/runbook
- Generator/Starter/Profile impact
- OpenAPI/JavaDoc/Test/Evidence/Guide

Source Trace

구분	Repository 경로	확인 포인트
Final Target	cpf-docs/governance/ CPF_FINAL_TARGET_REQUIREMENTS.md	180 canonical requirements
Architecture Guide	cpf-docs/architecture/ ARCHITECTURE_GUIDE.md	module/topology/failure/security
Starter Policy	cpf-docs/governance/ CPF_STARTER_ARCHITECTURE_AND_LIFECYCLE_POLICY.md	starter ownership/lifecycle
Settings	settings.gradle	physical module/profile closure
Routes	cpf-admin/frontend/src/app/routes.ts	control-plane capability

16. Architecture Decision Record 상세

ADR-01 Core Boot-free

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	cpf-core 는 Spring Boot 없는 계약 Artifact
Rationale	Public consumer 와 선택 Runtime 을 분리
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-02 Owner DB

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	한 데이터는 한 Owner 만 쓰기
Rationale	ADM/BZA/타 Domain 의 direct DB write 방지
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-03 Local/Remote parity

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	동일 Facade contract
Rationale	Topology 전환 시 업무 Source 재작성 방지
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-04 Spring Batch Primary

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	Job engine 중복 금지
Rationale	Spring Batch metadata/restart 생태계 재사용
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-05 Gateway ingress only

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	내부 service bus 로 사용하지 않음
Rationale	trust boundary 와 내부 latency/coupling 분리
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-06 Unknown first-class

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	응답 유실을 FAILED 로 단정하지 않음
Rationale	중복 side effect/금융 오처리 방지
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-07 Transaction strategy matrix

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	LOCAL/XA/Outbox/Inbox/Saga/TCC
Rationale	업무 일관성 요구에 따라 선택
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-08 Starter leaf ownership

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	Leaf→Profile→Aggregate→BOM
Rationale	Provider/footprint/conflict 통제
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-09 Product ADM/BZA vs EDU

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	제품 기능을 sample 이 복제하지 않음
Rationale	운영 기능의 단일 정보
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-10 Generator-first DB

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	Canonical schema→vendor packs
Rationale	3 Vendor drift 축소
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-11 Tamper-evident audit

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	hash chain + optional signature
Rationale	감사 변조 탐지
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-12 Secret provider/KMS/HSM

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	Key 원문을 application/config 에서 분리
Rationale	rotation/revocation/provider health
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-13 Browser BFF

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	token/session raw browser 노출 금지
Rationale	credential leakage 감소
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-14 Durable remote correlation

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	instance-local reply queue 금지
Rationale	multi-instance/rebalance 복구
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

ADR-15 Desired/actual operations

항목	내용
Context	금융/업무 플랫폼은 정상 경로뿐 아니라 topology/partial failure/security/operations 를 함께 다뤄야 한다.
Decision	version/checksum reconciliation
Rationale	partial apply/scale-out drift 통제
Trade-off	구현/운영 메타데이터가 늘어나지만 상태를 추측하지 않고 복구 가능성을 확보한다.
Failure impact	이 결정이 지켜지지 않으면 ownership drift, duplicate, unknown misclassification, stale operation 위험이 증가한다.
Verification	ArchUnit/build graph + actual consumer + fault/runtime scenario + documentation trace

적용 검토 질문

- Owner 와 Consumer 가 명확한가?
- Local/Remote/Multi-instance 에서 같은 의미인가?
- DB/Message/File/External side effect 를 어디서 확정하는가?
- 실패 시 durable state 와 recovery action 이 있는가?
- Permission/Masking/Approval/Audit 는 어떤 boundary 에서 집행하는가?

17. Capability Provider Ownership 보강

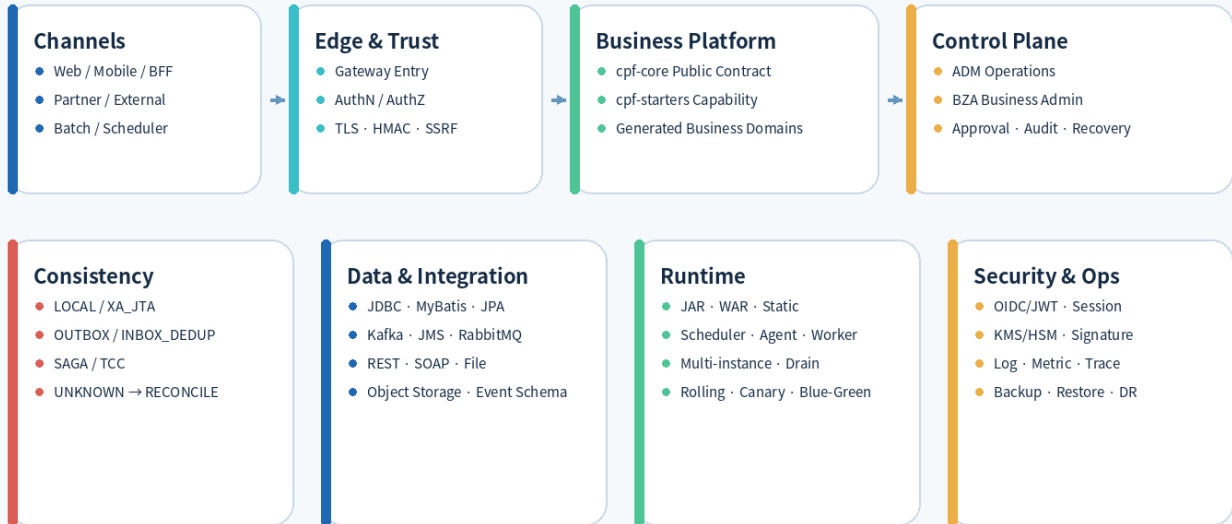
전역 Kernel 만 Core 에 두고 전용 Public Contract 와 선택 Runtime 은 실제 Capability/Owner/Leaf Starter 가 소유한다는 원칙을 현재 추가된 Provider 에 적용한 결과다.

Capability	Owner	책임	Consumer
Persistence Contract	cpf-core/api/persistence	provider-neutral CRUD/Search/Lock/Policy	Business Domain
JPA Runtime	cpf-starters/data/persistence-jpa	EntityManager convenience/timeout/observation	JPA 선택 Domain
Transaction Contract	cpf-core/api/transaction	strategy/definition/TCC/XA abstractions	Business/Batch
JTA Runtime	cpf-starters/data/transaction-jta	transaction manager adapter/XA datasource/recovery	XA 선택 Runtime
AI Contract	cpf-core/api/ai	provider-neutral request/response/risk/usage	AI Consumer
AI Runtime	cpf-starters/integration/ai	provider router/retry/circuit	AI 선택 Runtime
SOAP Runtime	cpf-starters/integration/soap	SOAP client/timeouts	External integration Consumer
OIDC Runtime	cpf-starters/security/oidc-login	login/claim mapping/user service	Browser/BFF/Product auth
Key/Signature	cpf-core/api/security/crypto + security/secret	provider-neutral key/sign contract + runtime routing	Security Consumer
Tamper Audit	cpf-core/api/security/audit + security/audit-jdbc	audit contract + JDBC store	ADM/BZA/security audit

19. 07_16 Architecture 확장 설계

CPF System Architecture

Channel부터 Control Plane · Data · Recovery까지 Owner와 Trust Boundary를 분리합니다.



CPF Architecture - Owner/Trust/Data/Recovery

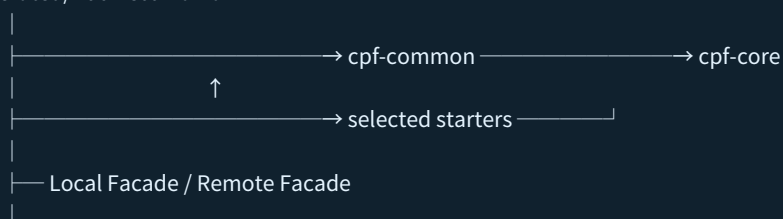
19.1 Core Slimming 과 Capability Ownership

Capability	Core 에 남는 것	Starter/Provider Owner	금지
Foundation Utility	policy/value contract	pure foundation/convenience	Core utility warehouse
Health	health meaning/port	platform-operations health	Actuator in Core
Lock	lock/lease/fence contract	JDBC/Valkey provider	provider impl in Core
Session	security/session contract	JDBC/Valkey session provider	auth fail-open
Object Storage	file/attachment contract	S3-compatible provider	AWS SDK in Public API
GraphQL	error/page/security contract	optional graphql starter	business logic in resolver
Realtime	event/progress contract	web realtime capability	unbounded fan-out

19.2 의존 방향

● ● ● CODE · ARCHITECTURE

Generated/Business Domain



Gateway / Batch / ADM / BZA → Public Contract only

cpf-core → cpf-starters/* = 0

19.3 Failure Boundary

● ● ● CODE · failure/recovery boundary

Local DB commit boundary

- |
- |— external call before side effect failure → FAILED / rollback
- |— external side effect success + response loss → UNKNOWN
- |— outbox commit + broker ACK loss → duplicate-safe publish
- |— XA prepare + process kill → TM recovery
- |— multi-instance lease loss → stale writer fenced

문서 기준 및 검증 범위

항목	기준
Repository / Branch	freeangelsun/202412_01_CPF / master
Work start / latest reviewed HEAD	64fd08d963927860e8d023403dfa276931801ee5 (07_17)
Latest product source change basis	4c4248a12e699c07f9f5fb11fbb33b97ca04077d (07_16 canonical currentization source)
Canonical Requirement	186 개
CURRENT SOURCE	실제 Repository 에서 확인한 Symbol/경로
PRODUCT CONTRACT	현재 구현량과 분리한 제품 목표 계약
REFERENCE	복사·응용 가능한 완성 기준 예시; CURRENT SOURCE 아님
Runtime qualification	문서 세션에서 직접 실행하지 않은 DB/Browser/Multi-instance/Fault 는 미검증

21. 07_17 Core Admission Architecture

Core dependency 는 “공통으로 보인다”가 아니라 아래 Admission Rule 을 모두 통과할 때만 허용합니다. 하나라도 실패하면 Capability/Owner/Provider/Starter 로 내려보냅니다.

Rule	Architecture Decision
CPF-wide	대부분 Capability 에 공통
No dedicated owner	Admin/Batch/Gateway/File/AI 등 전용 계약 아님
Non-optional	Optional 미사용 시에도 필요
Provider independent	Runtime/Topology/Provider 독립
Long-lived semantics	기술 교체 후 의미 유지
CPF kernel value	전역 Contract/Semantics/Value 또는 최소 Pure Logic

● ● ● TREE · Target dependency boundary

```

Business / Generated Domain
├── cpf-core (global kernel only)
├── owner-specific capability contract
└── selected provider starter

Fixed-Length
├── fixedlength-core (dedicated contract/engine)
└── fixedlength (Spring runtime)

Observability
├── core context semantics
├── platform-operations observability runtime
└── ADM control plane
  
```

[CURRENT SOURCE GAP] 07_16 Source 의 일부 provider-neutral 전용 계약이 아직 cpf-core 아래 존재합니다. Architecture Decision 은 07_17 정보를 기준으로 하며, 실제 package 이동 전까지 Source Trace 와 Target Owner 를 병기합니다.